

微光招新题 C-EASY-1

C-EASY-1

Part 1: 环境配置与调试

(1)gcc是gnu的编译器套件，是linux中最常用的c/c++编译器，mingw是windows的gcc移植版，让Windows能编译出.exe

(2)三个 json 文件的作用：

1.c_cpp_properties.json：给 VS Code 编辑器用的，程序纠错，不参与程序运行。

2.tasks.json：VS Code 通过 gcc 把 .c 代码转化为 .exe，使代码可以运行。

3.launch.json：给 gdb 用的，寻找程序，启动调试，控制弹外置黑框/内置终端。

(3)下载插件：获得语法高亮，代码补全，错误提示，编译运行及调试，提高写代码效率

(4)注释补全：{

```
// 使用 IntelliSense 了解相关属性。

// 悬停以查看现有属性的描述。

// 欲了解更多信息，请访问：https://go.microsoft.com/fwlink/?linkid=830387
"version": "0.2.0",

"configurations": [

  {

    "name": "gcc.exe - 生成和调试活动文件", // 该调试任务的名字，启动调试时会在待
    选列表中显示

    "type": "cppdbg",

    "request": "launch",

    "program": "${fileDirname}\\${fileBasenameNoExtension}.exe",
    "args": [],

    "stopAtEntry": false, //是否在程序入口处暂停

    "cwd": "${workspaceFolder}",

    "environment": [],

    "externalConsole": false, //是否使用外置控制台
```

```
"MIMode": "gdb",

"miDebuggerPath": "C:\\mingw64\\bin\\gdb.exe", // 调试器的路径

"setupCommands": [

    {

        "description": "为 gdb 启用整齐打印",

        "text": "-enable-pretty-printing",

        "ignoreFailures": true

    }

],

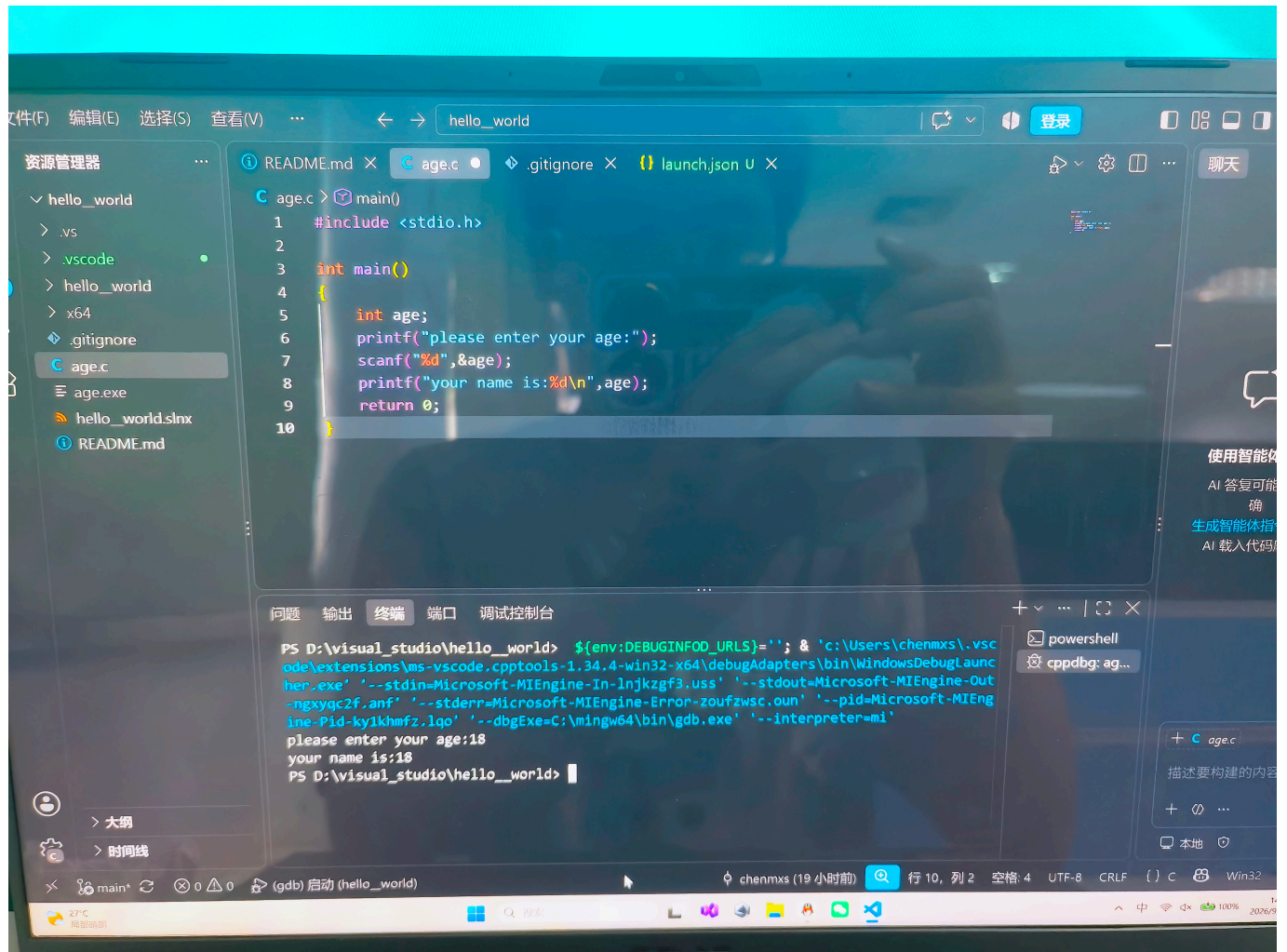
    "preLaunchTask": "C/C++: gcc.exe build active file" // 调试前的预执行任务，
    这里的值是tasks.json文件中的编译任务，也就是调试前需要先编译

}

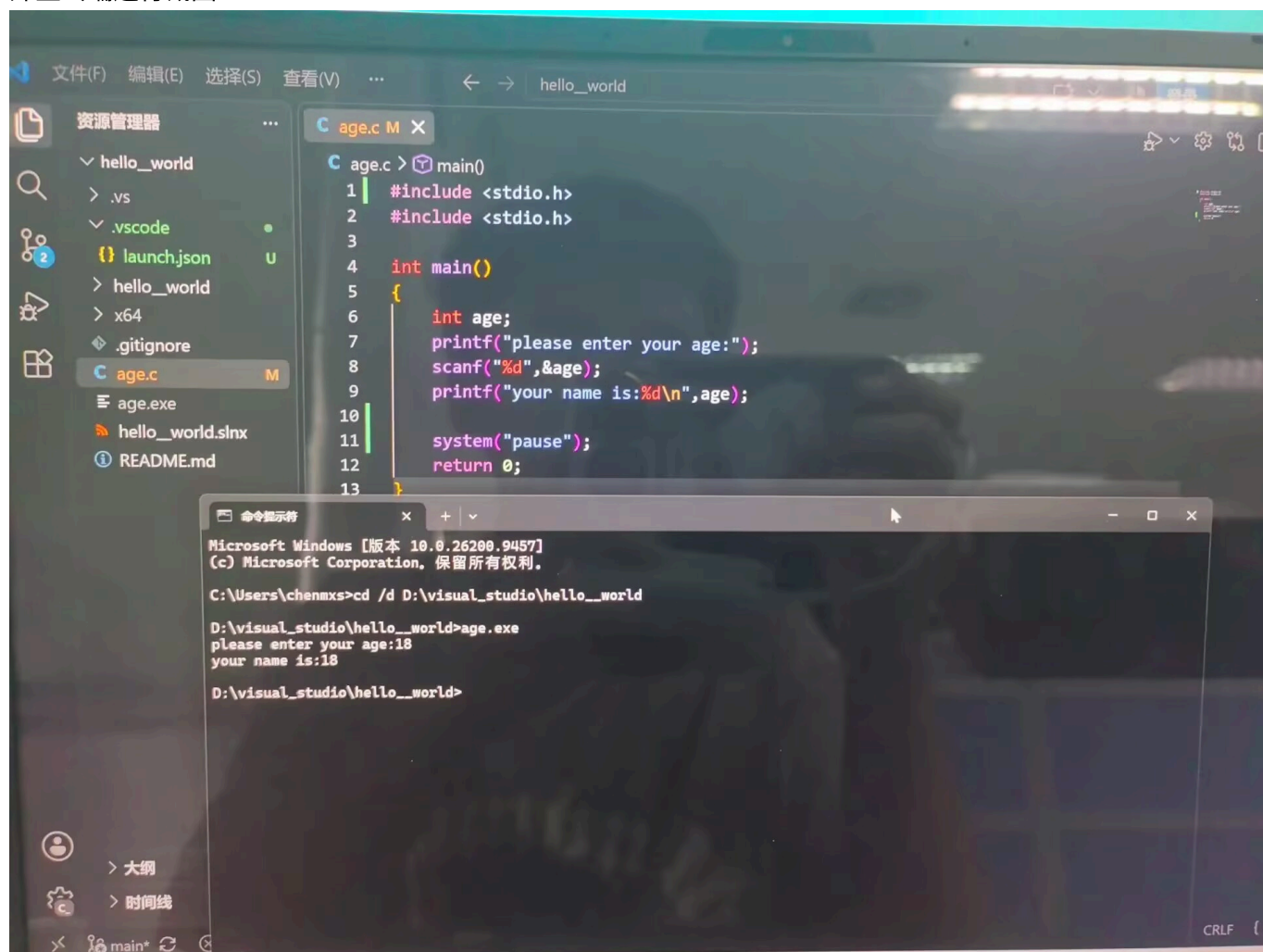
]
```

```
}
```

(5)内置终端运行截图：



外置终端运行截图：



Part 2: C语言基础

1. 理论题（用自己的话回答那 4 个问题：变量类型、数组边界、循环结构、逻辑表达式）

1. (1) 变量类型包括整型，实型，字符型，用于告诉电脑所装变量的数据类型，存储空间。(2) 重要：计算机存储内容固定，否则会出错。(3) 选择整数存放年龄。(4) 不能实现。如apple.c中演示通过字符数组可存储apple

```
#include <stdio.h> int main() { // char只能存单个字符 char a = 'x'; printf("%c\n", a); //字符数组可存单词 char b[]="apple"; printf("%s\n",b); return 0; }
```

2. (1) 数组第一个元素下标从0开始。(2) 超出有效范围程序仍会运行，可能读取垃圾信息，触发系统保护导致程序崩溃等。(3) 常见：c语言只会执行，不做边界检查。危险：如果修改超出有效范围的数组，可能会篡改挨着数组后的内存，导致程序崩溃。

3.(1)控制：判断条件是否满足，满足就继续进行循环，不满足结束进程。

基本结构：for(初始化{int i=? }, 条件判断{</>}, 迭代{i++/i--}){执行程序}

while(条件判断){执行程序}, do while先执行后判断(先循环一次)

(2)初始化：引入变量。 条件判断：将引入的变量与目标进行判断，满足则继续循环，即控制循环是否运行。
迭代：将变量向目标变化

(3)区别: while先判断后执行(与for一致), do while先执行后判断。

(4)见sum.c

```
#include <stdio.h> int main() { int sum=0; for(int i=0;i<=10;i++) { sum=sum+i; } printf("%d",sum); return 0; }
```

```
#include <stdio.h> int main() { int sum=0; int i=0; while(i<=10) { sum=sum+i; i++; } printf("%d",sum); return 0; }
```

· 相同点: 都能实现相同的重复计算逻辑, 代码执行结果完全一致 (都是 55)。

· 不同点: for 循环把初始化、条件判断、迭代三步都写在了同一行括号里, 结构紧凑, 适合已知循环次数的场景 (比如这里固定求 1 到 10) while 循环把初始化和迭代拆到了外面和循环体里, 结构更灵活, 适合循环次数未知、需要根据条件动态判断的场景。

4.(1)区别: 1.运算对象:算术表达式: 操作数值; 逻辑表达式: 操作布尔数(真/假)

2.运算结果: 算术表达式结果为数值; 逻辑表达式结果为真/假

(2)含义: '&&': 与, 都为真时结果为真

'||': 或, 满足一个结果为真

!:非, 真变假, 假变真(反转布尔值/表达“不”)(区分! =不等于)

(3)示例与预测:

- 假设: `int age = 20; int score = 75;`
- 表达式: `(age >= 18) && (score >= 60)`
- 预测结果: 真。因为 20 大于等于 18 是真, 75 大于等于 60 也是真, “真 && 真”结果为真。
- 表达式: `(age < 18) || (score > 90)`
- 预测结果: 假。因为两个条件都是假, “假 || 假”结果为假。

2. 编程题 (代码文件: part2.c (输入姓名年龄、计数、询问是否继续))

见part2.c

```
#include <stdio.h>
```

```
int main() { int age; char name[20]; int count =0; char choice;
```

```
while(1)
{
    printf("请输入你的年龄");
    scanf("%d",&age);
    printf("请输入你的名字");
    scanf("%s",name);
    printf("年龄: %d,姓名: %s",age,name);
    count++;
    printf("是否继续");
```

```

        scanf(" %c",&choice);
        if(choice=='n')
        {
            printf("总共输入了%d次",count);
            break;
        }
    }
    return 0;
}

```

Part 3: 函数

1. 函数重构 代码文件：part3.c（封装乘方、综合成绩计算、排序打印）

```
#include <stdio.h>
```

```
//1.方差 int power(int base,int exp) { int result =1; for(int i=0;i<exp;i++) { result =result *base; } return result; }
```

```
//2.综合成绩 int score(int a,int b,int c) { int p=(a+b+c)/3; int f=(power(p-a,2)+power(p-b,2)+power(p-c,2))/3;
return 3*p-f/3 ; }
```

```
//3排序打印 void sort(int zh1,int zh2,int zh3) { if (zh1 >= zh2 && zh2 >= zh3) { printf("小明 > 小强 > 小林"); }
else if (zh1 >= zh3 && zh3 >= zh2) { printf("小明 > 小林 > 小强"); } else if (zh2 >= zh1 && zh1 >= zh3) {
printf("小强 > 小明 > 小林"); } else if (zh2 >= zh3 && zh3 >= zh1) { printf("小强 > 小林 > 小明"); } else if (zh3
>= zh1 && zh1 >= zh2) { printf("小林 > 小明 > 小强"); } else { // zh3 >= zh2 && zh2 >= zh1 printf("小林 > 小
强 > 小明"); } }
```

```
int main() { int x1, x2, x3; int y1, y2, y3; int z1, z2, z3;
```

```

printf("请输入小明的三项成绩(顺序为A B C,以一个空格为间隔) : ");
scanf("%d %d %d", &x1, &x2, &x3);
printf("请输入小强的三项成绩(顺序为A B C,以一个空格为间隔) : ");
scanf("%d %d %d", &y1, &y2, &y3);
printf("请输入小林的三项成绩(顺序为A B C,以一个空格为间隔) : ");
scanf("%d %d %d", &z1, &z2, &z3);

```

```

int zh1 =score(x1,x2,x3);
int zh2 =score(y1,y2,y3);
int zh3 =score(z1,z2,z3);
sort(zh1,zh2,zh3);

```

```
return 0;
```

```
}
```

2. 值传递问题（回答为什么用普通变量传参的 swap 会失败，以及应该怎么用指针修改）

(1)结果：交换失败，原因：调用swap(a,b)时实参调用数据交给形参，不会使main中真正的a，b交换

(2)真正交换：需使用指针传递变量的地址原函数中a, b改为*a, *b, 调用方式改为swap(&a,&b)

Day1

今日完成

·成功尝试通过git上传文件

·独立写了一个helloworld程序

Day2

今日完成（括号内是ai辅助）

·三个json

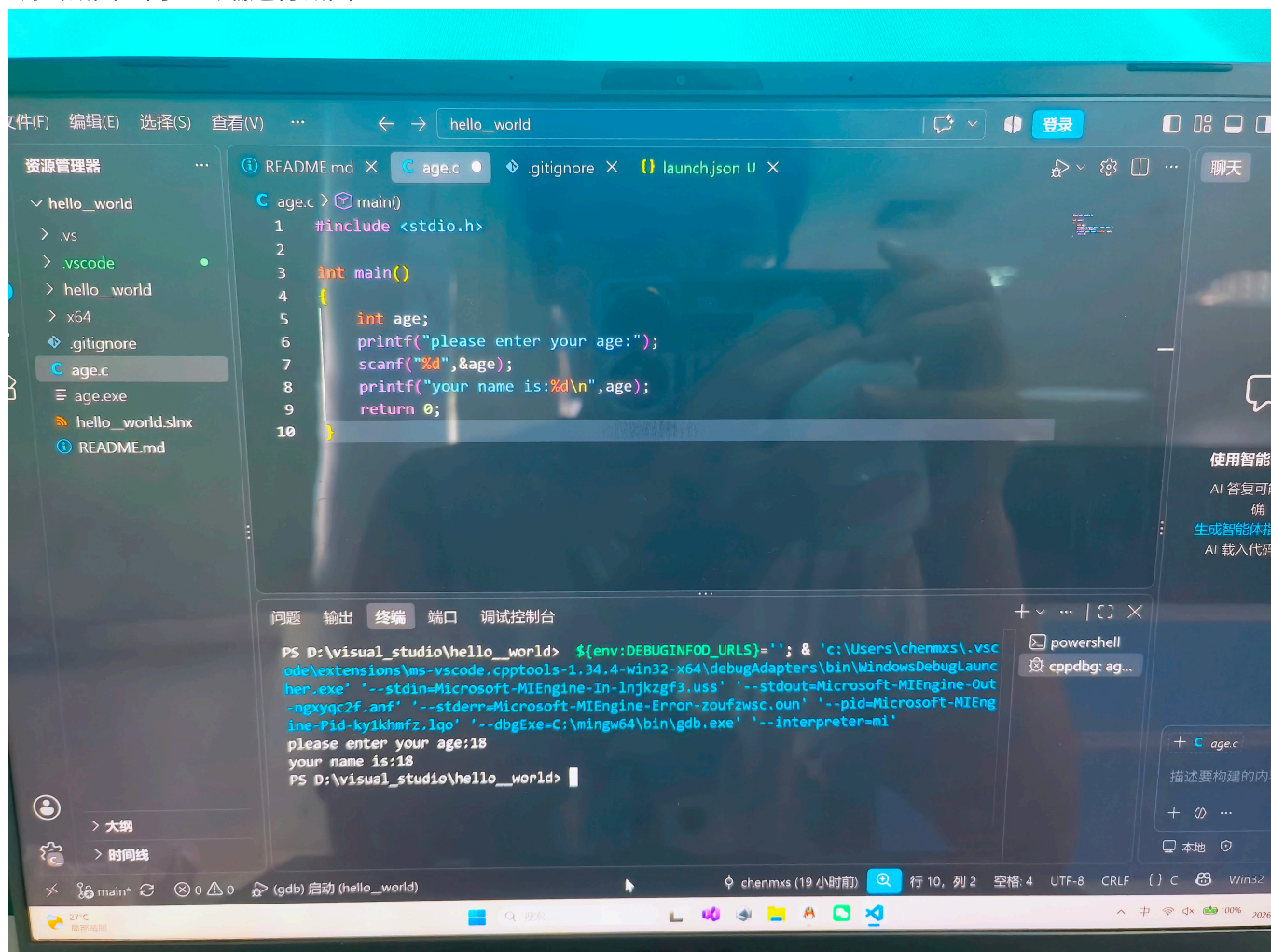
(1) c_cpp_properties.json,作用：(给vscode编辑器用的) 程序纠错，不参与程序运行

(2) tasks.json,作用：(vscode通过gcc)把.c代码转化为.exe格式，使代码可以运行

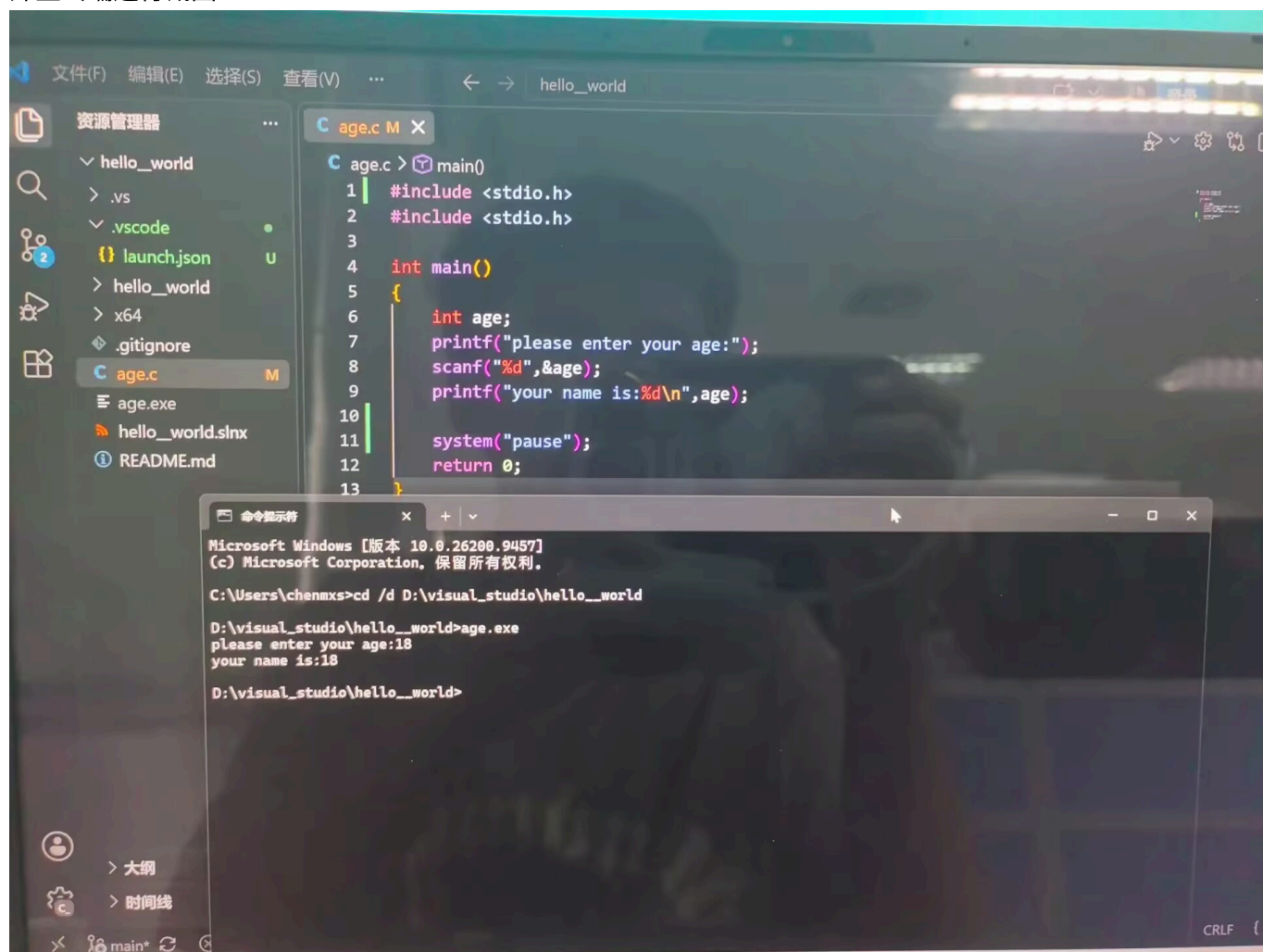
(3) launch.json,作用：(给gdb用的) 寻找程序，启动调试，控制弹外置黑框/内置终端

·独立完成九九乘法表，调试踩坑：printf () 直接在双引号里写i*j，结果全是i, j，不会换行

·调试截图：内置终端运行截图：



外置终端运行截图：



Day 3

今日完成

- 看完了视频 `if/else`、`while`、`switch` 分支语句。
- 独立手敲并跑通了三个程序：
 - `score.c`：成绩等级判断（练 `if/else if` 多分支）。
 - `leap-year.c`：闰年判断（练逻辑运算符 `&&` 和 `||` 的优先级）。
 - `calculator.c`：简易计算器（练 `switch...case` 分支和 `break`）。

踩坑记录

- if连写错误**：不能写 `100 >= number >= 90`，必须用 `&&` 拆开，如 `number >= 90 && number <= 100`。
- switch穿透**：`case` 里的 `printf` 后面必须加 `break;`，否则会穿透执行下面的所有 `case`。